UNIVERSIDAD DE LA SABANA

Facultad de Ingeniería

Ingeniería Informática

TALLER UNIFICADO — HERRAMIENTAS DE BIG DATA

Pipeline de Datos sobre YouTube Trending Dataset

2026

1. Introducción

El presente informe documenta el desarrollo del Taller Unificado de Herramientas de Big Data, realizado en el marco del programa de Ingeniería Informática de la Universidad de La Sabana. El objetivo central fue diseñar e implementar un pipeline de datos completo sobre el dataset público de tendencias de YouTube (datasnaek/youtube-new), disponible en la plataforma Kaggle.

El taller se estructuró en tres partes progresivas e interconectadas:

- Parte I: Construcción del pipeline de ingesta, transformación y orquestación de datos, haciendo uso de Prefect 3 como herramienta de orquestación y Pandas como librería de procesamiento.
- Parte II: Construcción de un dashboard de visualización interactivo con Plotly, que consume los datos procesados en la Parte I y genera un archivo HTML autocontenido con ocho gráficos.
- Parte III: Integración del pipeline con MongoDB Atlas como motor de persistencia NoSQL, complementado con análisis georreferenciado y de correlación estadística mediante un Jupyter Notebook.

El dataset de YouTube Trending registra los videos que aparecieron en la sección de tendencias de YouTube para diez países: Estados Unidos (US), Reino Unido (GB), Canadá (CA), Alemania (DE), Francia (FR), Rusia (RU), México (MX), Corea del Sur (KR), Japón (JP) e India (IN). El periodo analizado comprende principalmente los años 2017 y 2018.

2. Parte I — Pipeline de Ingesta y Transformación

2.1. Arquitectura general del pipeline

El pipeline fue implementado en Python 3.11 y orquestado con Prefect 3, un framework de orquestación de flujos de trabajo que permite definir flows (flujos) compuestos por tasks (tareas), con soporte nativo para reintentos automáticos, logging estructurado y ejecución en modo efímero (sin necesidad de servidor dedicado).

El flujo principal se llama youtube_trending_pipeline y articula cuatro etapas en secuencia:

1. Creación de la zona de staging local (directorios staging/youtube_raw/ y staging/youtube_processed/).
2. Descarga y carga de los archivos CSV desde Kaggle mediante la API oficial.
3. Estandarización de fechas por dos métodos distintos con verificación cruzada de resultados.
4. Cálculo de métricas de temporalidad y persistencia del dataset final procesado.

2.2. Ingesta de datos (Tarea a)

La descarga de los archivos se realizó a través de la librería kaggle, que autentica con la API oficial de Kaggle usando un archivo kaggle.json. El pipeline contempla un mecanismo de resiliencia importante: si las credenciales no están disponibles o falla la descarga, la función _generar_datos_sinteticos() crea archivos CSV con la misma estructura del dataset real (mismas columnas, formatos de fecha idénticos, distribuciones plausibles de vistas y likes), lo que permite probar el pipeline en entornos sin acceso a internet.

La tarea descargar_dataset_kaggle está configurada con retries=2 y retry_delay_seconds=10, lo que significa que Prefect reintentará la descarga hasta dos veces antes de caer en el modo sintético.

Tras la carga inicial de los diez archivos CSV, se obtuvo el siguiente resultado:

Resultado	Valor
Filas totales cargadas	239,662
Países con encoding UTF-8 válido	6 (US, GB, CA, DE, FR, IN)
Países omitidos por error de encoding	4 (JP, KR, MX, RU)
Filas efectivamente procesadas	238,662

Los archivos de Japón, Corea del Sur, México y Rusia contienen caracteres fuera del rango UTF-8 (caracteres asiáticos y cirílicos), por lo que fueron descartados en esta iteración. La tarea cargar_y_unificar_paises utiliza el parámetro on_bad_lines='skip' para manejar líneas con formato incorrecto dentro de cada archivo, y captura excepciones de lectura por archivo con un bloque try/except, registrando una advertencia en el log de Prefect sin interrumpir el flujo.

2.3. Unificación y selección del Top 100 global (Tarea b)

Una vez cargados los DataFrames de cada país, se añadió una columna country a cada uno con el código de país (US, GB, etc.) y se concatenaron en un único DataFrame unificado usando pd.concat. Posteriormente se aplicaron dos operaciones de reducción:

- Eliminación de duplicados por video_id: un mismo video puede aparecer en la lista de tendencias de varios países. Se tomó la primera aparición (la del país con mayor número de vistas registrado para ese video_id).
- Ordenamiento descendente por views y selección de los primeros 100 registros (TOP_N = 100).

El dataset final del Top 100 global tiene las siguientes características:

Métrica	Valor
Total de registros	100 videos
Total de columnas	25
Mínimo de vistas para entrar al Top 100	36,957,773
Video más visto	Nicky Jam x J. Balvin - X (EQUIS)
Vistas del #1	424,538,912
País de origen del registro #1	Reino Unido (GB)

La distribución por país de origen de los 100 registros es la siguiente:

País	Código	Videos en el Top 100
Reino Unido	GB	83
Estados Unidos	US	13
India	IN	3
Canadá	CA	1

La concentración de registros en Gran Bretaña (83 de 100) no implica que los videos sean de artistas británicos, sino que el dataset de GB es el que registró los mayores valores de views para estos videos globales. El contenido dominante en el Top 100 corresponde a videoclips musicales de artistas latinoamericanos (Bad Bunny, Nicky Jam, J. Balvin, Ozuna, Daddy Yankee) y de pop anglosajón (Drake, Ariana Grande, Childish Gambino), además del video YouTube Rewind 2017.

La distribución por categoría de contenido es la siguiente:

Categoría	Videos	% del Top 100
Music	82	82%
Entertainment	11	11%
Film & Animation	3	3%
People & Blogs	1	1%
Howto & Style	1	1%
Comedy	1	1%
Science & Tech	1	1%

2.4. Estandarización de fechas (Tarea c)

El dataset original presentó dos campos de fecha con formatos no estándar que requirieron transformación explícita:

- **trending_date**: formato propio del dataset, YY.DD.MM (ejemplo: 17.14.11 = 14 de noviembre de 2017). Este formato es propenso a confusión con ISO 8601 y no es reconocido automáticamente por Pandas.
- **publish_time**: formato ISO 8601 con zona horaria UTC (ejemplo: 2018-03-02T05:00:19.000Z). Requería eliminación del timezone para homogeneizar el tipo datetime.

Se implementaron dos métodos independientes para la estandarización, con el objetivo de comparar enfoques y validar la consistencia de los resultados:

Método 1 — apply + lambda (vectorizado)

La tarea estandarizar_fechas_apply_lambda aplica las funciones auxiliares _parsear_trending_date y _parsear_publish_time sobre cada columna usando el método .apply() de Pandas. Este enfoque es más eficiente en memoria y velocidad para datasets de gran tamaño, ya que opera sobre la columna completa sin iteración fila a fila explícita. Los resultados se almacenan en las columnas trending_date_std y publish_time_std.

Método 2 — iterrows (iteración explícita)

La tarea estandarizar_fechas_iterrows itera fila a fila sobre el DataFrame usando el método .iterrows() de Pandas, aplicando las mismas funciones auxiliares de parseo. Los resultados se almacenan en columnas separadas (trending_date_std_iter y publish_time_std_iter) para permitir la comparación directa con el Método 1.

Tras la ejecución de ambos métodos, el flow verifica con dos assertions explícitas que los resultados son idénticos:

```
assert df['trending_date_std'].equals(df['trending_date_std_iter'])
assert df['publish_time_std'].equals(df['publish_time_std_iter'])
```

Ambas assertions pasaron sin error (NaT: 0 en los cuatro campos resultantes), confirmando que los dos métodos producen exactamente el mismo resultado. Esto es relevante porque valida tanto la corrección del parseo como la equivalencia funcional de los dos enfoques de transformación.

2.5. Cálculo de duración hasta tendencia (Tarea d)

La tarea calcular_duracion_tendencia calcula, para cada video del Top 100, cuánto tiempo transcurrió desde su publicación (publish_time_std) hasta la fecha en que apareció como tendencia (trending_date_std). La diferencia se calcula como un objeto timedelta de Python/Pandas y se convierte a tres representaciones:

- **delta_tendencia**: diferencia como timedelta (ejemplo: 35 days 18:59:41).
- **dias_hasta_tendencia**: parte entera de días (accedida via .dt.days).
- **horas_hasta_tendencia**: duración total en horas (via .dt.total_seconds() / 3600).

Adicionalmente, cada video fue etiquetado con una categoria_velocidad según la duración calculada:

Categoría	Criterio	Videos	% del Top 100
viral_inmediato	≤ 1 día	0	0%
rapido	2 – 7 días	4	4%
moderado	8 – 30 días	47	47%
lento	> 30 días	49	49%

Los estadísticos descriptivos completos de la variable dias_hasta_tendencia son:

Estadístico	Valor
Media	26.69 días
Desviación estándar	9.79 días
Mínimo	5 días
Percentil 25 (Q1)	17 días
Mediana (Q2)	30 días
Percentil 75 (Q3)	35 días
Máximo	37 días

El hallazgo más relevante de esta etapa es que ningún video del Top 100 global alcanzó la categoría viral_inmediato, y solo 4 videos (4%) lograron viralizarse en la primera semana. Casi la mitad del Top 100 (49 videos) tardó más de 30 días en aparecer como tendencia. Esto sugiere que los videos con mayor volumen acumulado de vistas no son necesariamente los que generan mayor impacto inmediato, sino los que mantienen una audiencia sostenida durante varias semanas.

3. Parte II — Dashboard Interactivo con Plotly

3.1. Descripción general

La segunda parte del taller consistió en la construcción de un dashboard de visualización interactivo que consume el archivo youtube_top100_global.csv generado por la Parte I, junto con los archivos JSON de categorías de YouTube (category_id.json) almacenados en staging/youtube_raw/. El dashboard fue implementado en Python usando la librería Plotly y genera como salida un archivo HTML autocontenido (dashboard_youtube.html) que puede visualizarse en cualquier navegador sin dependencias adicionales, ya que incluye el motor de Plotly embebido desde CDN.

El script parte_2_dashboard.py también respeta la estructura de orquestación de la Parte I: sus funciones están diseñadas para poder ser decoradas con @flow y @task de Prefect 3 con mínimas modificaciones, manteniendo coherencia arquitectónica con el pipeline principal.

3.2. Procesamiento de datos para visualización

Antes de construir las figuras, la función cargar_datos() enriquece el DataFrame con columnas derivadas necesarias para las visualizaciones:

- **category_name**: nombre legible de la categoría (Music, Entertainment, etc.), resuelto a partir del campo numérico category_id mediante el diccionario CAT_MAP.
- **country_name**: nombre completo del país (United Kingdom, United States, etc.) resuelto desde el código de país.
- **polaridad**: diferencia entre likes y dislikes (likes − dislikes), que mide el sentimiento neto de la audiencia.
- **polaridad_pct**: porcentaje de polaridad positiva calculado como (likes − dislikes) / (likes + dislikes + 1) × 100.
- **title_short**: versión truncada a 48 caracteres del título, para uso en etiquetas de gráficos.

Los nombres de categoría también se enriquecen dinámicamente leyendo los archivos JSON de Kaggle mediante la función cargar_categorias_json(), que itera sobre todos los archivos *_category_id.json y construye un diccionario unificado {category_id: category_name}.

3.3. Visualizaciones implementadas

El dashboard incluye 8 visualizaciones interactivas organizadas en un layout de dos columnas (grid CSS). Todas comparten un tema visual oscuro consistente (fondo #0d1117, tipografía Space Mono/DM Sans) y admiten interacciones de zoom, pan y tooltip enriquecido.

#	Título de la visualización	Tipo de gráfico	Alcance
1	Top 20 Vídeos Más Vistos (Global)	Barras horizontales por categoría	Ancho completo

#	Título de la visualización	Tipo de gráfico	Alcance
2	Vistas Totales y Videos por País	Barras + línea en eje secundario	Mitad
3	Interacción por País (Vistas/Likes/Dislikes/Polaridad)	Barras agrupadas + línea	Mitad
4	Vistas por Categoría y País (datos JSON)	Heatmap con valores en millones	Ancho completo
5	Grado de Interacción por Zona y Categoría	Scatter de burbujas (tamaño=vistas, color=polaridad)	Ancho completo
6	Polaridad de Interacción por Categoría	Barras horizontales divergentes	Mitad
7	Distribución Días hasta Tendencia	Violin plot con puntos individuales	Mitad
8	Jerarquía de Vistas: País › Categoría	Treemap jerárquico interactivo	Ancho completo

3.4. KPIs del encabezado

El dashboard presenta en su parte superior una fila de KPIs calculados directamente desde el DataFrame del Top 100. Los valores reales obtenidos son:

KPI	Valor
Vistas totales acumuladas (Top 100)	8,704,590,629 (~8.7B)
Likes totales acumulados	127,789,845 (~127.8M)
Dislikes totales acumulados	10,312,917 (~10.3M)
Polaridad global	85.1%
País con mayor volumen de vistas	United Kingdom (GB)
Categoría líder en vistas	Music
Promedio de días hasta tendencia	27 días

3.5. Decisiones de diseño

Se definió un esquema de colores por categoría consistente a lo largo de todas las visualizaciones (por ejemplo, Music siempre aparece en #58a6ff, Entertainment en #f78166), lo que facilita la lectura comparativa entre gráficos distintos. El archivo HTML final utiliza un layout CSS Grid de dos columnas con soporte responsive (una sola columna en pantallas menores a 900px) y referencia la librería de Plotly desde el CDN

oficial (plotly-2.35.2.min.js), lo que garantiza portabilidad sin instalar dependencias adicionales en el equipo del receptor.

4. Parte III — Integración con MongoDB y Análisis

4.1. Descripción general

La tercera parte del taller fue desarrollada en un Jupyter Notebook (parte3.ipynb) y extendió el pipeline para incluir tres funcionalidades adicionales: persistencia de datos en MongoDB Atlas, visualización georreferenciada mediante mapas coropléticos y análisis estadístico de correlación.

4.2. Almacenamiento en MongoDB Atlas

La integración con la base de datos se implementó usando la librería pymongo y la orquestación de Prefect 3. Se estableció una conexión con MongoDB Atlas (servicio cloud de MongoDB) y los 100 registros del dataset procesado fueron almacenados en la colección top100 de la base de datos youtube_db.

El flujo de carga (pipeline_parte3) sigue el siguiente proceso:

5. Se lee el archivo youtube_top100_global.csv con Pandas.
6. La tarea guardar_mongo convierte el DataFrame a una lista de diccionarios Python usando df.to_dict('records').
7. Se ejecuta collection.delete_many({}) para vaciar la colección antes de la carga, garantizando idempotencia: si el pipeline se ejecuta múltiples veces, siempre refleja el estado más reciente del CSV procesado.
8. Se inserta la lista de documentos con collection.insert_many(data).

Tras la carga, se verificó el almacenamiento consultando collection.count_documents({}) (resultado esperado: 100) y recuperando un documento de ejemplo con collection.find_one().

La elección de MongoDB como motor de persistencia es coherente con la naturaleza del dataset: los registros de YouTube Trending contienen campos con valores compuestos (el campo tags puede contener listas de etiquetas delimitadas por pipe) y la variabilidad entre países hace que un esquema documental sea más flexible que uno relacional estricto.

4.3. Visualización georreferenciada

Se implementaron mapas coropléticos mundiales utilizando la función px.choropleth de Plotly Express, con el parámetro locationmode='country names' para mapear los códigos de país del dataset a regiones geográficas. Las dos visualizaciones georreferenciadas generadas fueron:

Categoría más vista por país

Para cada país del dataset, se agruparon las vistas por category_id, se identificó la categoría con mayor volumen acumulado y se representó en el mapa. El código implementado es el siguiente:

```
cat_top = df.groupby(['country', 'category_id'])['views'].sum().reset_index()
cat_top = cat_top.loc[cat_top.groupby('country')['views'].idxmax()]
fig = px.choropleth(cat_top, locations='country', locationmode='country names',
                    color='category_id', title='Categoría más vista por país')
```

Esta visualización permite identificar si existen patrones de consumo diferenciados por región o si la dominancia de la categoría Music es uniforme entre los países representados en el dataset.

4.4. Análisis de correlación

Se calculó el coeficiente de correlación de Pearson entre la variable dias_hasta_tendencia (tiempo desde publicación hasta aparecer en tendencias) y la variable views (número de reproducciones):

```
correlacion = df['dias_hasta_tendencia'].corr(df['views'])
print('Correlación:', correlacion)
```

El valor obtenido fue de 0.226, lo que indica una correlación positiva débil entre ambas variables. Esto significa que, en el Top 100, los videos que tardaron más días en hacerse tendencia tienden a tener marginalmente más vistas, aunque la relación no es estadísticamente fuerte.

Esta correlación positiva débil es coherente con el hallazgo de la Parte I: los videos más vistos no son los que viralizan de forma inmediata, sino los que acumulan audiencia de manera sostenida durante semanas. Un video que permanece en tendencias durante más tiempo acumula más vistas, lo que puede explicar la correlación positiva observada.

Para visualizar la relación, se generó un scatter plot con línea de regresión OLS (Ordinary Least Squares) mediante Plotly Express:

```
fig = px.scatter(df, x='dias_hasta_tendencia', y='views',
                 trendline='ols',
                 title='Relación entre tiempo en tendencia y vistas')
```

La línea de regresión OLS permite observar la tendencia general de la nube de puntos, confirmando visualmente la correlación positiva calculada.

5. Tecnologías y Herramientas Utilizadas

Tecnología / Librería	Uso en el taller
Python 3.11	Lenguaje principal de desarrollo en las tres partes del taller
Prefect 3	Orquestación de flows y tasks, manejo de reintentos y logging estructurado
Pandas	Carga, transformación, agrupación y análisis del dataset
Kaggle API (kaggle)	Descarga automatizada del dataset datasnaek/youtube-new
Plotly / Plotly Express	Visualizaciones interactivas: barras, heatmaps, scatter, treemap, violin, choropleth
pymongo	Conexión y operaciones CRUD sobre MongoDB Atlas
MongoDB Atlas	Base de datos NoSQL en la nube para persistencia de los 100 documentos procesados
Jupyter Notebook	Entorno de desarrollo interactivo para la Parte III
Miniconda / Anaconda	Gestión de entornos virtuales Python

La estructura de archivos del proyecto es la siguiente:

```
Taller_Unificado_HBD/
├── parte_1/
│   ├── youtube_pipeline.py       # Pipeline principal con Prefect 3
│   └── informe_parte1.md         # Hallazgos de la Parte I
├── parte_2/
│   ├── parte_2_dashboard.py      # Script generador del dashboard
│   └── dashboard_youtube.html    # Dashboard HTML generado
├── parte_3/
│   ├── parte3.ipynb              # Notebook con MongoDB + análisis
│   ├── youtube_pipeline.py       # Pipeline reutilizado en Parte III
│   └── informe_parte3.md         # Hallazgos de la Parte III
└── staging/
    ├── youtube_raw/              # CSVs originales de Kaggle
    └── youtube_processed/
        └── youtube_top100_global.csv  # Dataset final (100 filas, 25 columnas)
```

6. Conclusiones

6.1. Hallazgos sobre los datos

Dominio de la música en el Top 100. El 82% de los 100 videos más vistos corresponde a la categoría Music, con una concentración marcada en artistas de reggaetón y trap latino (Nicky Jam, J. Balvin, Bad Bunny, Ozuna, Daddy Yankee) y pop anglosajón (Drake, Ariana Grande, Childish Gambino). Esto refleja el perfil de consumo global en YouTube durante 2017-2018.

La viralización sostenida supera a la inmediata. Ningún video del Top 100 alcanzó la categoría viral_inmediato (≤ 1 día). Solo el 4% llegó a tendencias en los primeros 7 días. El 96% restante tardó entre 8 y 37 días, con una mediana de 30 días. Esto indica que el mayor volumen de vistas acumuladas está asociado a videos que mantienen presencia en tendencias durante semanas, no a impactos virales instantáneos.

Correlación positiva débil entre duración y vistas. El coeficiente de correlación de Pearson entre dias_hasta_tendencia y views fue de 0.226, confirmando una relación positiva pero débil. Los videos que permanecen más tiempo acumulando tendencia tienden a tener marginalmente más vistas, lo que es consistente con el hallazgo anterior.

Alta polaridad positiva global. La polaridad global del Top 100 es del 85.1%, calculada como (likes − dislikes) / (likes + dislikes). Esto indica que el contenido más visto genera una respuesta predominantemente positiva de la audiencia.

6.2. Aprendizajes técnicos

Prefect 3 como orquestador. El framework demostró ser adecuado para pipelines de datos de escala media, ofreciendo reintentos automáticos configurables, logging estructurado integrado y compatibilidad con ejecución local sin infraestructura adicional (modo efímero). La separación en tasks facilita la reutilización de componentes entre partes del taller.

Validación cruzada de transformaciones. Implementar la estandarización de fechas con dos métodos distintos (apply+lambda e iterrows) y verificar su equivalencia mediante assertions explícitas es una práctica robusta de ingeniería de datos que garantiza la corrección del dato transformado antes de persistirlo.

MongoDB para datos semiestructurados. La integración con MongoDB Atlas fue directa gracias a pymongo. El patrón delete_many + insert_many garantiza idempotencia, lo que es crítico en pipelines que pueden ejecutarse múltiples veces sobre el mismo dataset de origen.

6.3. Trabajo futuro

- Corregir el encoding de los archivos de Japón, Corea del Sur, México y Rusia (posiblemente encoding='latin-1' o 'cp1251' para el archivo ruso) para incluir los 10 países en el análisis y enriquecer el Top 100 global con diversidad geográfica.
- Implementar un scheduler con Prefect para mantener el dataset actualizado periódicamente, consumiendo versiones más recientes del dataset de Kaggle o conectando con la YouTube Data API v3 directamente.
- Extender el análisis estadístico con variables adicionales (likes_per_view, comment_count, días de permanencia en tendencias) y construir un modelo de regresión múltiple para predecir las vistas esperadas de un video.
- Añadir un sistema de alertas en Prefect (notificaciones por email o Slack) cuando el pipeline falle o cuando el Top 100 cambie significativamente respecto a la ejecución anterior.